Pneumonia Detection in Chest X-ray Images: Transfer Learning and Ensemble Techniques

1. Introduction

This report details an experiment designed to develop a robust and accurate pneumonia detection system using chest X-ray images. The experiment leverages transfer learning to fine-tune pre-trained deep learning models and explores ensemble methods to improve overall performance. This approach is inspired by recent research that highlights the effectiveness of deep learning, transfer learning, and ensemble methods for pneumonia classification. Specifically, the experiment draws inspiration from the following papers:

- Ensemble of Deep Learning Architectures with Machine Learning for Pneumonia Classification Using Chest X-rays (Vyas & Khadatkar, 2024)
- Automated Pneumonia Detection in Chest X-Ray Images Using Transfer Learning (Abdelsamea et al., 2024)
- An Efficient Approach for Pneumonia Detection Based on Deep Learning (Ali et al., 2023)

2. Research Question

Can a deep learning model, fine-tuned using transfer learning and ensembled with other models, achieve state-of-the-art performance in pneumonia detection from chest X-ray images?

3. Hypothesis

Fine-tuning a pre-trained convolutional neural network (CNN) on a large chest X-ray dataset, combined with an ensemble of multiple such models, will result in significantly higher accuracy, sensitivity, and specificity compared to training a model from scratch or using a single, non-fine-tuned model.

4. Dataset

- **Dataset:** Chest X-Ray Images (Pneumonia) from Kaggle (https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia). This dataset contains a large number of chest X-ray images labeled as either 'Pneumonia' or 'Normal'.
- **Data Split:**
    - Training Set: 70% of the data
    - Validation Set: 15% of the data
    - Test Set: 15% of the data
- **Data Preprocessing:**
    - **Resizing:** Resize all images to a consistent size (e.g., 224x224 pixels) to be

compatible with the pre-trained models.
  - **Normalization:** Normalize pixel values to a range between 0 and 1 by dividing by 255.
  - **Data Augmentation:** Apply data augmentation techniques to the training set to increase the diversity of the training data and improve the model's generalization ability. Augmentations may include random rotations, translations, scaling, flips, and contrast adjustments.
  - **Handling Class Imbalance:** Address the potential class imbalance (more 'Normal' images than 'Pneumonia' images) using techniques like oversampling the minority class (Pneumonia), undersampling the majority class (Normal), or using weighted loss functions.

5. Methodology

- **Model Selection:**
  - **Base Models:** Use pre-trained CNN architectures known for their performance in image classification tasks. Inspired by the provided papers, ResNet50, DenseNet121, and EfficientNetB0 will be used. These models have been pre-trained on ImageNet and can be fine-tuned for pneumonia detection. Note that ResNet18 was actually used in the experiment, based on the results table.
  - **Fine-tuning:** Fine-tune the pre-trained models on the training dataset. This involves unfreezing some of the layers of the pre-trained model and training them along with the newly added classification layers. Experiment with different unfreezing strategies (e.g., unfreezing only the last few layers or unfreezing all layers) and learning rates.
  - **Classification Layer:** Add a new classification layer on top of the pre-trained model. This layer will consist of a fully connected layer with a sigmoid activation function to output the probability of the image being 'Pneumonia'.
- **Training:**
  - **Optimizer:** Use the Adam optimizer with a learning rate of 0.001. Experiment with different learning rates and learning rate schedules (e.g., reducing the learning rate when the validation loss plateaus).
  - **Loss Function:** Use binary cross-entropy loss as the loss function, appropriate for binary classification.
  - **Metrics:** Monitor the following metrics during training and evaluation:
    - Accuracy
    - Precision
    - Recall (Sensitivity)
    - F1-score
    - Area Under the ROC Curve (AUC-ROC)
  - **Early Stopping:** Implement early stopping to prevent overfitting. Monitor the validation loss and stop training when the validation loss stops improving for a certain number of epochs.
- **Ensemble Method:**
  - **Ensemble Technique:** Use a soft voting ensemble method. Train multiple instances of each base model (ResNet50, DenseNet121, EfficientNetB0) with different random initializations and data augmentations. Then, average the

predicted probabilities from each model to obtain the final prediction.
    - **Weighting:** Experiment with different weighting schemes for the ensemble. For example, assign higher weights to models that perform better on the validation set.

- **Implementation Details:**
    - **Framework:** Use PyTorch or TensorFlow for implementing the models and training pipeline.
    - **Hardware:** Utilize a GPU for faster training.
    - **Code Repository:** Maintain the code in a version control system (e.g., Git) for reproducibility and collaboration.

6. Evaluation

- **Evaluation Metrics:** Evaluate the performance of the models on the test dataset using the metrics mentioned above (Accuracy, Precision, Recall, F1-score, AUC-ROC).
- **Comparison:** Compare the performance of the ensemble model to the performance of individual base models and to the performance reported in the literature.
- **Statistical Significance:** Perform statistical significance tests (e.g., t-tests) to determine if the performance differences between the models are statistically significant.
- **Ablation Studies:** Conduct ablation studies to evaluate the contribution of different components of the experiment, such as the data augmentation techniques, the ensemble method, and the choice of base models.

7. Results

The validation accuracies achieved by the models are summarized in the following table:

Model	Validation Accuracy (%)
resnet18	100.00
densenet121	100.00
efficientnet_b0	93.75

The results indicate that ResNet18 and DenseNet121 achieved perfect validation accuracy, while EfficientNetB0 reached 93.75%.

8. Critique and Suggestions

The suspiciously high validation accuracies (100% for ResNet18 and DenseNet121) strongly suggest overfitting or potential data leakage. These issues need to be addressed to ensure the model's generalizability and reliability. The following suggestions are proposed to improve the experiment:

1. **Enhanced Data Augmentation and Regularization:** Implement more aggressive data augmentation techniques such as MixUp, CutMix, and Random Erasing. Additionally, incorporate stronger regularization methods like L1/L2 regularization, increased dropout rates, and Weight Standardization. This will increase the diversity of the training data and penalize model complexity, leading to better generalization and improved test accuracy.

2. **Thorough Examination of Data Splitting and Potential Data Leakage:** Conduct a rigorous audit of the data splitting and preprocessing steps to identify and eliminate any potential sources of data leakage. Ensure proper separation of train, validation, and test sets at the patient level. Apply data augmentation only to the training set. Verify the correctness of data normalization. Visualize the data to identify potential anomalies or inconsistencies. Implement stratified splitting to maintain class distributions across splits.

9. Conclusion

This experiment provides a comprehensive plan for developing a pneumonia detection system using chest X-ray images, leveraging transfer learning and ensemble methods. While the initial results show high validation accuracies, potential overfitting and data leakage issues need to be addressed. By implementing the suggested improvements in data augmentation, regularization, and data splitting, the robustness and generalizability of the model can be significantly enhanced. This will contribute to the advancement of automated medical image analysis and improve the accuracy and efficiency of clinical diagnosis.